

Unit 1

SMD Practical + Oral Exam Notes

UNIT 1 — INTRODUCTION TO OOM AND UML

Topic 1: Object Oriented Methodologies

1. Simple Exam Definition:

Methodologies are structured approaches or sets of rules used to analyze, design, and build software using OO concepts. The four major pre-UML methodologies are Booch, Rumbaugh (OMT), Jacobson (OOSE), and Coad Yourdon.

2. Core Concept Explanation:

- **Booch:** Focuses on design and implementation. Strong on class and object diagrams. Used distinctive "cloud" shapes for objects.
- **Rumbaugh (OMT):** Object Modeling Technique. Uses 3 models — Object (static structure), Dynamic (states and events), Functional (data flow).
- **Jacobson (OOSE):** Object-Oriented Software Engineering. Introduced **Use Cases** as the central concept to capture user requirements and drive the entire design.
- **Coad Yourdon:** Simpler methodology, good for prototyping, focused on notations for classes and relationships.

3. Real Practical Example (Student Management System):

- **Jacobson's influence:** "Student enrolls in a Course" is a Use Case. This drives every design decision.
- **Rumbaugh's influence:** We model the `Student` object with attributes (name, ID) and its static relationships with `Course`.
- **Booch's influence:** We design `Student` class with methods like `enroll()`, `viewGrades()`.

4. Examiner Viva Questions:

- What is OOSE? Who introduced it?
- What are the three models in OMT?
- Who introduced Use Cases?

5. Strong Viva Answers:

- *Answer:* OOSE (Object-Oriented Software Engineering) was introduced by Ivar Jacobson. It is use-case driven — requirements are expressed as use cases.
- *Answer:* OMT by Rumbaugh uses three models: **Object Model** (static structure), **Dynamic Model** (states), **Functional Model** (data flow/DFD).
- *Answer:* Use Cases were introduced by **Ivar Jacobson** and are now the backbone of UML requirement modeling.

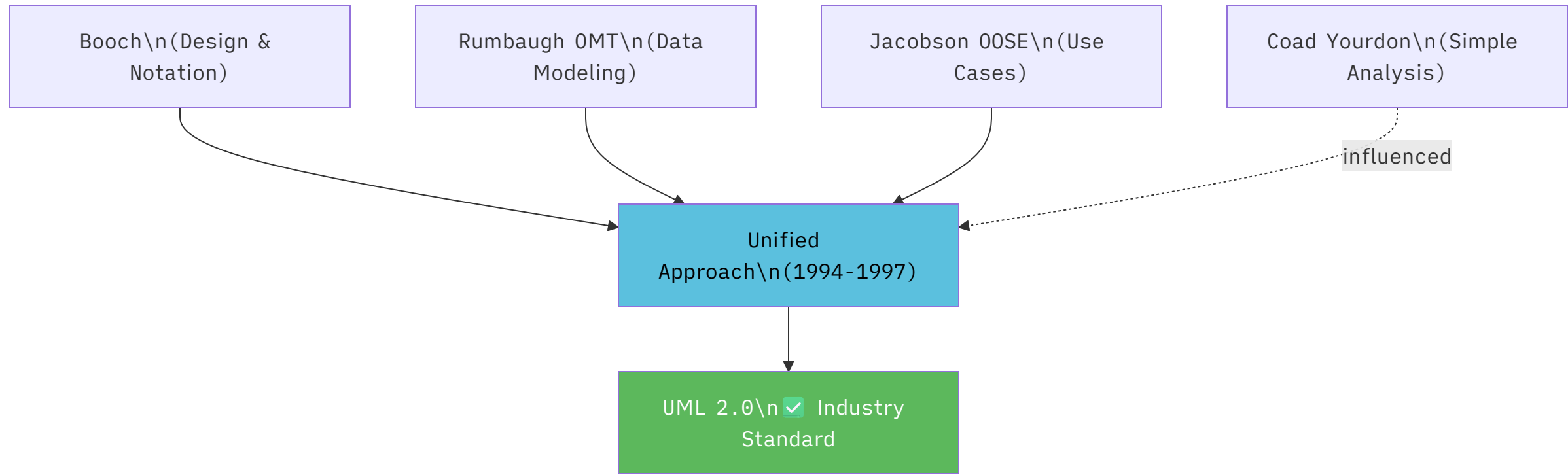
6. Common Mistakes Students Make:

- Saying Booch invented Use Cases. (Wrong — Jacobson did.)
- Not knowing that all three methodologies merged into UML.
- Confusing OMT models — Dynamic model is NOT the same as a Sequence diagram.

9. Difference Tables:

Methodology	Author	Key Focus	Key Contribution
Booch	Grady Booch	Design & Implementation	Rich class/object notation
OMT	James Rumbaugh	Data Modeling	Object, Dynamic, Functional models
OOSE	Ivar Jacobson	User Requirements	Use Cases
Coad Yourdon	Peter Coad	Simple Analysis	Easy notation for prototyping

📊 Visual: How UML Was Born



10. Memory Tricks / Shortcuts:

- Booch = **B**uild (Design/Implementation) .
- Rumbaugh = **R**elationships (Data Modeling) .
- Jacobson = **J**ust Use Cases .
- All Three Authors = **The Three Amigos** of UML .

12. Oral Trap Questions:

- *Examiner:* "Which methodology is the best?"
- *Answer:* "None alone is sufficient. That's exactly why Booch, Rumbaugh, and Jacobson (the Three Amigos) unified their methods to create UML — each contributed something the others lacked."

Topic 2: Unified Approach, OOA, OOD, Iterative Development

1. Simple Exam Definition:

- **Unified Approach (UA):** Combines OOA + OOD in a use-case-driven, iterative process using UML.
- **OOA:** Identifies what the system should do (real-world objects, use cases).
- **OOD:** Defines how the system will do it (design classes, interfaces, architecture).
- **Iterative Development:** Build the system in small cycles (iterations), each producing a working piece of software.

2. Core Concept Explanation:

OOA vs OOD — The core difference:

- **OOA** = Problem Domain. You study the real world and model it as objects. Output: Use Case Diagrams, Object Model.
- **OOD** = Solution Domain. You design the solution using classes, methods, databases. Output: Class Diagrams, Deployment Diagrams.

Iterative Development:

Instead of building everything at once (Waterfall), you build in **iterations**:

- Iteration 1: Login + Enrollment module (working, testable).
- Iteration 2: Grade Management module.
- Iteration 3: Report Generation module.

Each iteration includes Analysis → Design → Code → Test. This is how modern software is built (Agile is based on this).

Continuous Testing:

Testing happens in EVERY iteration, not just at the end. This catches bugs early and reduces cost.

3. Real Practical Example (Student Management System):

- **Iteration 1:** Build Login + Course Enrollment. Show it to the client. Get feedback.
- **Iteration 2:** Build Grade Calculation based on feedback from Iteration 1.
- **Iteration 3:** Build Admin Reports and Deployment.

4. Examiner Viva Questions:

- What is the difference between OOA and OOD?
- What is Iterative Development? Why is it better than Waterfall?
- Can we do OOD without OOA?

5. Strong Viva Answers:

- *Answer:* OOA focuses on "what" the system does (Problem Domain). OOD focuses on "how" the system does it (Solution Domain). OOA output becomes the input for OOD.
- *Answer:* Iterative Development builds software in small cycles, each delivering working functionality. It is better than Waterfall because testing and feedback happen continuously, reducing risk.
- *Answer:* No. Without OOA, we don't know what objects and use cases exist, so OOD would have nothing to design from.

6. Common Mistakes Students Make:

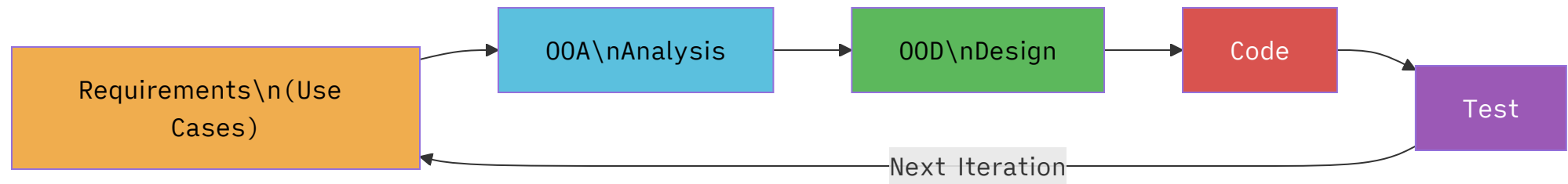
- Saying UA and UML are the same. (UA is the process. UML is the notation/language.)
- Confusing Iterative with Incremental. (Iterative = refine each cycle. Incremental = add new features each cycle.)

9. Difference Tables :

Feature	OOA	OOD
Goal	Understand the problem	Create the solution
Focus	"What"	"How"
Domain	Real World (Problem)	Computer World (Solution)
Output	Use Cases, Objects	Classes, Interfaces, DB Schema

Feature	Iterative	Waterfall
Testing	Every iteration	Only at end
Feedback	Early and continuous	Very late
Risk	Low	High
Changes	Easy to accommodate	Very expensive

Visual: Iterative Development Cycle



10. Memory Tricks / Shortcuts:

- **A**nalysis = **A**sk "What?" (Real world) .
- **D**esign = **D**ecide "How?" (Computer world) .
- Iterative = Like cooking: taste and improve each step.

Topic 3: UML 2.0 — Building Blocks, Diagrams Overview & Notational Techniques

1. Simple Exam Definition:

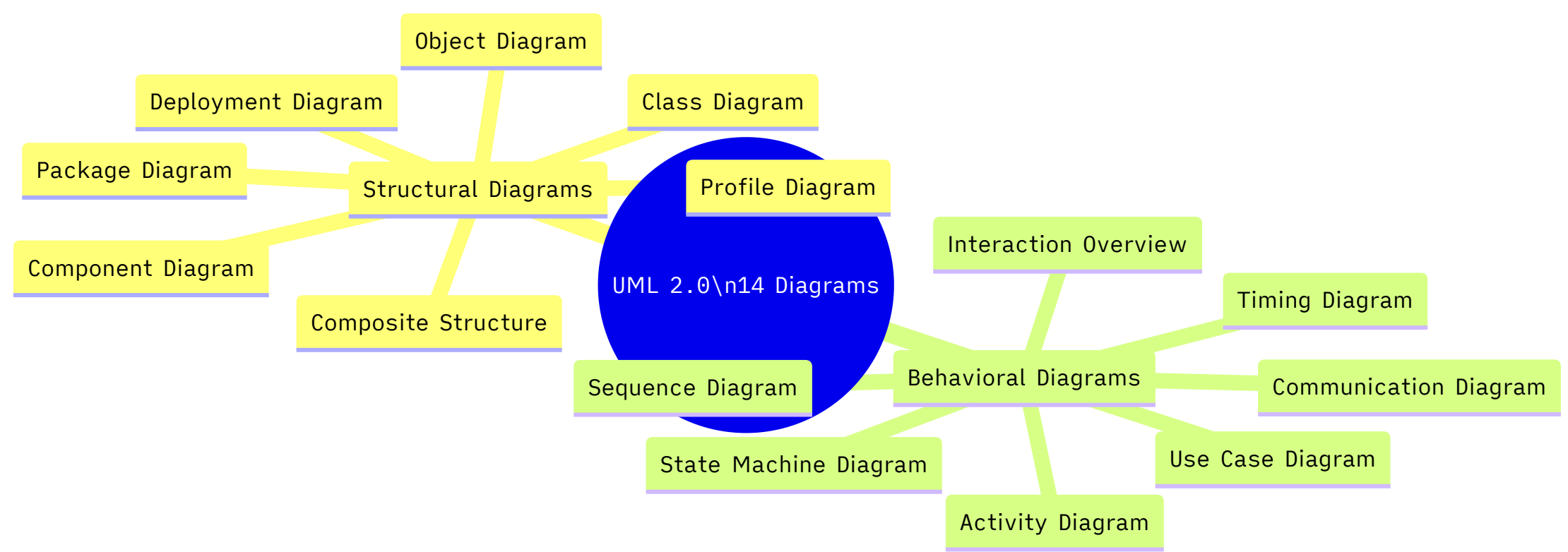
UML (Unified Modeling Language) is a standard visual language used to model software systems. It consists of 14 diagram types organized under Building Blocks: Things, Relationships, and Diagrams.

2. Core Concept Explanation — UML Building Blocks:

- **Things (4 types):**
 - *Structural*: Classes, Interfaces, Components, Nodes.
 - *Behavioral*: Use Cases, Interactions, State Machines.
 - *Grouping*: Packages.

- *Annotational*: Notes/Comments.
- **Relationships (4 types)**:
 - *Association*: General link (solid line).
 - *Generalization*: Inheritance (solid line + hollow triangle).
 - *Dependency*: "Uses" (dashed arrow).
 - *Realization*: Implements interface (dashed + hollow triangle).
- **Diagrams (14 total, split into 2 categories)**:

 Visual: UML 2.0 — All 14 Diagrams

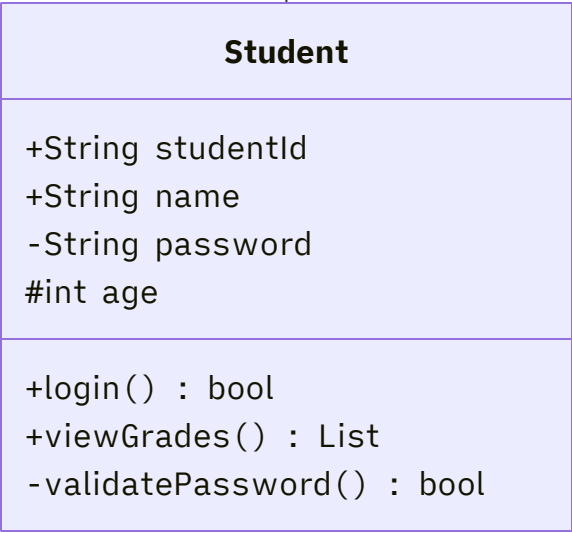


3. UML Notational Techniques — What Examiners Check:

Element	Shape	Notes
Class	Rectangle (3 compartments)	Name / Attributes / Methods
Actor	Stick Figure	Outside system boundary
Use Case	Oval	Inside system boundary
Association	Solid line	With/without arrow
Generalization	Solid + Hollow triangle	Arrow points to parent
Aggregation	Solid + Hollow diamond	Diamond at parent
Composition	Solid + Filled diamond	Diamond at whole
Dependency	Dashed arrow	Points to dependency
Note	Rectangle with folded corner	Can be attached to any element

 Visual: Class Box Notation

Visibility: \n+ = public\n- = private\n# = protected\n~ = package



4. UML Common Mechanisms:

UML uses 4 common mechanisms across all diagrams:

- 1. **Specifications:** Behind every element is a specification (text description of semantics).
- 2. **Adornments:** Extra details added to elements (e.g., multiplicity `1..*` on lines).
- 3. **Common Divisions:** Classifier vs Instance (Class vs Object), Interface vs Implementation.
- 4. **Extensibility:** Stereotypes (`<<interface>>`), Tagged Values, Constraints `{ordered}`.

4. Examiner Viva Questions:

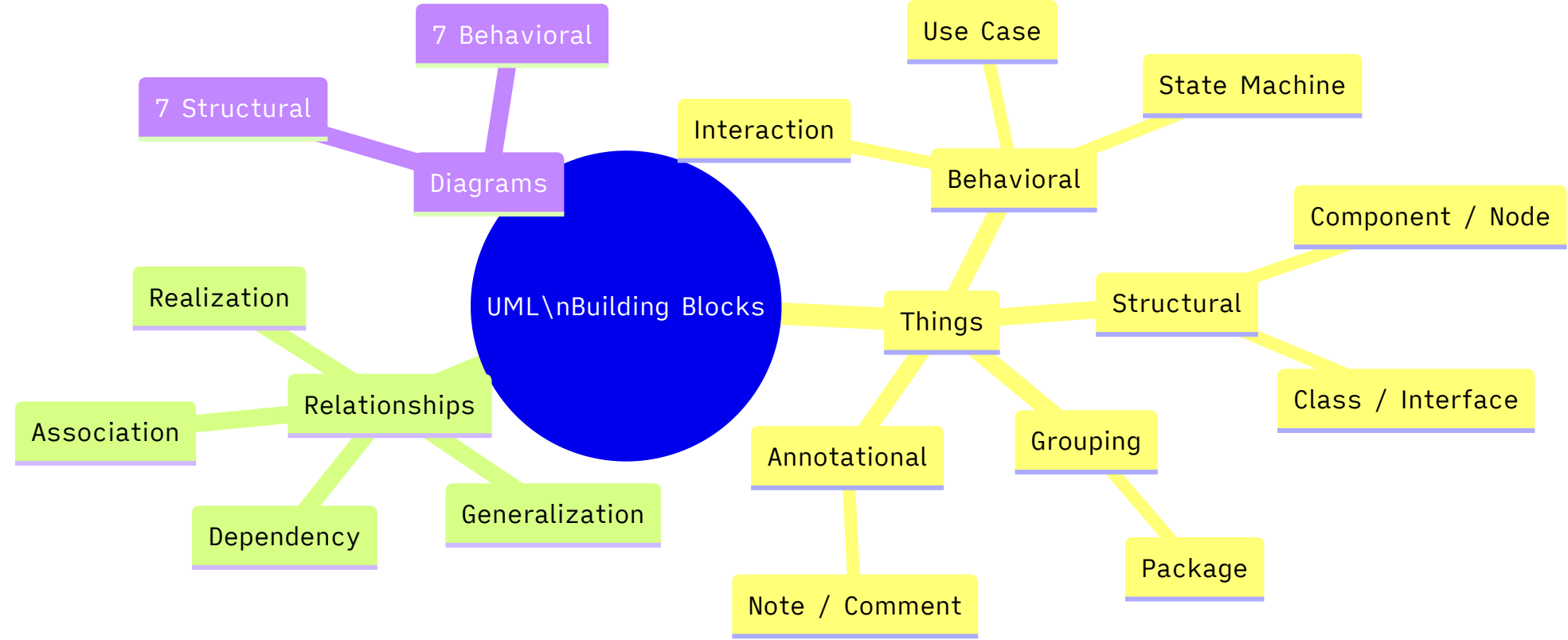
- Name all 3 UML Building Blocks.
- How many types of diagrams are in UML 2.0?
- What is the difference between Structural and Behavioral diagrams?
- What are UML Stereotypes? Give an example.

5. Strong Viva Answers:

- *Answer:* UML 2.0 has **14 diagrams**: 7 Structural and 7 Behavioral.
- *Answer:* Structural diagrams show the static structure (what exists). Behavioral diagrams show dynamic behavior (what happens).
- *Answer:* A stereotype is a UML extensibility mechanism that classifies an element. Example: `<<interface>>`, `<<include>>`, `<<actor>>`.

6. Common Mistakes Students Make:

- Saying UML has only 9 or 10 diagrams. (It has 14 in UML 2.0.)
- Not knowing that Class and Deployment diagrams are Structural, while Sequence and Use Case are Behavioral.



9. Difference Tables :

Diagram Type	Static/Dynamic	Key Purpose
Class Diagram	Static	Show classes & relationships
Use Case Diagram	Static	Show actor-system interactions
Sequence Diagram	Dynamic	Show time-ordered messages
Activity Diagram	Dynamic	Show workflow / control flow
State Diagram	Dynamic	Show object lifecycle
Deployment Diagram	Static	Show hardware deployment
Component Diagram	Static	Show software modules

10. Memory Tricks :

- "CUD SOA" for the 7 most tested diagrams: **C**lass, **U**se Case, **D**eployment, **S**equence, **O**bject, **A**ctivity, State.
- Structural = **Static** (the skeleton) . Behavioral = **Dynamic** (the movement) .

Topic 4: 4+1 View Model

1. Simple Exam Definition :

The 4+1 View Model (by Philippe Kruchten) describes a software system from 5 different architectural perspectives, with the Use Case View at the center driving all others.

2. Core Concept Explanation :

View	Focus	UML Diagram Used
+1 Use Case View	Actors & system behavior (center)	Use Case Diagram
Logical View	Classes, objects, functionality	Class Diagram, Object Diagram
Process View	Concurrency, performance, threads	Activity Diagram, Sequence Diagram
Implementation View	Source code, components	Component Diagram
Deployment View	Physical hardware and deployment	Deployment Diagram

Why 4+1 and not 5?

The Use Case View is **not** counted as one of the 4 because it is the *central connecting view* that drives the other 4. It is the "+1".

3. Real Practical Example (Student Management System):

- **+1 Use Case View:** Actors (Student, Professor, Admin) + Use Cases (Enroll, Login, Grade).
- **Logical View:** `Student` class, `Course` class, `GradeCalculator` class.
- **Process View:** How enrollment request flows through the system (Sequence diagram).
- **Implementation View:** `student-module.jar`, `db-connector.jar` components.
- **Deployment View:** Client Browser → Web Server → DB Server.

4. Examiner Viva Questions:

- Why is it called 4+1 and not 5?
- Which UML diagram corresponds to the Logical View?
- Who proposed the 4+1 View Model?

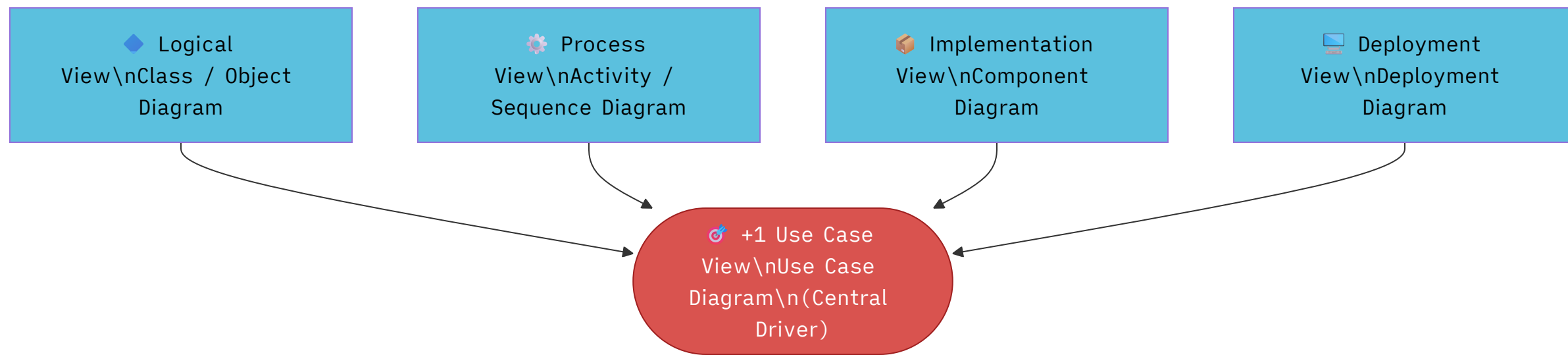
5. Strong Viva Answers:

- *Answer:* 4+1 because the **Use Case View** is the architectural driver — it unifies and connects the other 4 views, not counted as a peer.
- *Answer:* Philippe Kruchten proposed the 4+1 View Model. It separates concerns so different stakeholders see only the view relevant to them.

6. Common Mistakes Students Make:

- Saying there are only 4 views. (Always say "4 + **1** = 5 views total.")
- Placing the wrong diagram in the wrong view.

📊 Visual: 4+1 View Model



💡 **Exam tip:** In viva, say: "The Use Case View is the architectural backbone. All design decisions in the other 4 views are traceable back to a use case."

10. Memory Tricks:

- **L**ogical = **L**ook at classes.
- **P**rocess = **P**erformance & threads.
- **I**mplementation = **I**nstallable components.
- **D**eployment = **D**evelopes & hardware.
- **+1 Use Case** = The **boss** view.

11. High Probability Questions:

- Explain the 4+1 View Model with a diagram and give one UML diagram for each view.